

12.6 RAG Queries

For our question to lead to a meaningful answer, several steps are executed in succession by the LlamaIndex library:

1. The question in natural language is converted into vectors. For this purpose, the same embedding model must be used as when creating `VectorStoreIndex`.
2. The system searches for similar vectors in the database.
3. The content of the LlamaIndex nodes found is redirected to an LLM as a context with the question.
4. The LLM formulates an answer in natural language from the data.

Having dealt with the embedding model so far, this is the first time we've come into contact with the LLM. But that's not quite true: in [Section 12.4](#), `TitleExtractor` used an LLM to create a summary from the split documents. Which LLM is used, and where can we configure it?

LlamaIndex currently uses OpenAI's `gpt-4o` as the default LLM, provided that you've stored a valid API key in the `OPENAI_API_KEY` environment variable. However, you can change this behavior very easily and use a local `llama3.1` from your Ollama server, for example:

```
from llama_index.core.settings import Settings
from llama_index.llms.ollama import Ollama
...
Settings.llm = Ollama(model="llama3.1")
```

Alternatively, you can also use the popular Claude 3.5 Sonnet:

```
from llama_index.llms.anthropic import Anthropic
...
```

```
Settings.llm = Anthropic(model="claude-3-5-sonnet-20240620")
```

Note the import of `settings` and the subsequent `Settings.llm` setting. LlamaIndex now uses the set model for all accesses to an LLM. The same applies to the embedding model. The setting for this is `Settings.embed_model`.

Prompt: Which butterfly species was found most frequently in Tyrol in 2023?

To ask this question, we still need a LlamaIndex `query_engine`, which we can derive from the `VectorStoreIndex`:

```
query_engine = index.as_query_engine()
response = query_engine.query(
    "Which butterfly species was found "
    "most frequently in Tyrol in 2023?"
)
print(response)
```

The different answers from the various models are fascinating. The preceding question refers to a table on page 41 of the 2023 annual report on insect monitoring in Austria. The PDF document contains a wide range of information on the scientific results, not only on the investigations in Tyrol. Here is the answer from the combination of the `bge-m3` embedding model and the Claude 3.5 Sonnet LLM:

The large ox-eye butterfly (*Maniola jurtina*) was observed most frequently in Tyrol in 2023. With 430 individuals counted, it was the species with the most sightings that year.

The information is even more detailed when we use the OpenAI embedding model:

The large ox-eye butterfly (*Maniola jurtina*) was the butterfly species most frequently observed in Tyrol in 2023. This species remains at the top of the list with the most individuals counted. This is followed in frequency by the small cabbage white butterfly (*Pieris rapae*), the common blue butterfly

(*Polyommatus icarus*), and the small meadow bird (*Coenonympha pamphilus*).

However, it's important to note that the small fox (*Aglais urticae*) was the most widespread species and was detected at 87 out of 100 sites surveyed.

If you've ever wanted to extract data from a table in a PDF document yourself, then you'll know that in most cases, it's faster to enter the data again. PDF describes where something is on a page, but not how it belongs together in terms of content. This makes it all the more astonishing how well the combination of vector search and LLM works. The fact that Claude points out that the small fox was found at most locations is remarkable, even if the number 87 refers to the entire period between 2018 and 2023 and not to the sought-after year 2023.

You can download the PDF document from the project website to get your own impression of the initial data at <https://vielfalter.at/ergebnisse/veroeffentlichungen>.

12.6.1 Source Code

After all the code snippets, we want to show you the complete program that we used to test the models. It contains command-line parameters for selecting the embedding model and the LLM as well as a switch to regenerate the `VectorStoreIndex` and a switch for more logging output. We haven't printed the many `import` statements as they aren't necessary to understand the program.

Note that when creating the `VectorStoreIndex`, the value of the `args.embed` switch is transferred as the name of the *collection*. If this switch isn't specified, the `openai` default value will be used. This allows you to save several indexes in parallel in the Milvus database and test them in parallel.

One more note on the environment variables for OpenAI and Anthropic: we've created an `.env` file in the current directory with the following content:

```
OPENAI_API_KEY="sk-proj-dG...."
ANTHROPIC_API_KEY='sk-ant-api03-....'
```

The Python library `dotenv`, which you can install using `pip install dotenv`, loads this file. Provided you still have some credit left in your accounts, you can try out the various models from OpenAI or Anthropic.

```
# File: vf-rag.py
load_dotenv()
parser = argparse.ArgumentParser(
    prog="vf-rag",
    description="Information about viel-falter.at
project")
parser.add_argument("-v", "--verbose",
    action="store_true")
parser.add_argument("-d", "--debug",
    action="store_true")
parser.add_argument("-r", "--regenerate",
    action="store_true")
parser.add_argument(
    "-l", "--llm", choices=["ollama", "openai",
"claude"],
    default="claude")
parser.add_argument(
    "-e", "--embed", default="openai",
    choices=["m3", "nomic", "mxbai", "base",
"openai"])
args = parser.parse_args()

if args.verbose or args.debug:
    logging.basicConfig(
        stream=sys.stdout,
        level=logging.DEBUG if args.debug else
logging.INFO)
    logging.getLogger().addHandler(logging.StreamHandler(
        stream=sys.stdout))

milvus_store = "http://localhost:19530"
dimensions = 1024
if args.embed == "m3":
    Settings.embed_model = HuggingFaceEmbedding(
```

```

        model_name="BAAI/bge-m3",
        cache_folder="/home/user/huggingface_cache")
elif args.embed == "base":
    Settings.embed_model = HuggingFaceEmbedding(
        model_name="BAAI/bge-base-en-v1.5",
        cache_folder="/home/user/huggingface_cache")
    dimensions = 768
elif args.embed == "nomic":
    Settings.embed_model = OllamaEmbedding(
        model_name="nomic-embed-text")
    dimensions = 512
elif args.embed == "mxbai":
    Settings.embed_model = OllamaEmbedding(
        model_name="mxbai-embed-large")
else:
    Settings.embed_model = OpenAIEmbedding()
    dimensions = 1536

if args.llm == "ollama":
    Settings.llm = Ollama(model="llama3.1")
elif args.llm == "openai":
    Settings.llm = OpenAI(model="gpt-4o-mini")
else:
    Settings.llm = Anthropic(
        model="claude-3-5-sonnet-20240620")

logging.log(level=logging.INFO, msg=args)
logging.log(level=logging.DEBUG, msg=os.environ)
if args.regenerate:
    pdf_documents = SimpleDirectoryReader("pdfs").load_data(
        show_progress=True)
    webpages = [
        "https://viel-falter.at/",
        "https://viel-falter.at/monitoring/ziele-
vision/",
        "https://viel-falter.at/projektpartner/",
        "https://viel-falter.at/news/",
        "https://viel-falter.at/tagfalter-exkursion/",
    ]
    web_documents = SimpleWebPageReader(
        html_to_text=True).load_data(webpages)
    documents = pdf_documents + web_documents

```

```

vector_store = MilvusVectorStore(
    uri=milvus_store, dim=dimensions, overwrite=True,
    collection_name=args.embed)
storage_context = StorageContext.from_defaults(
    vector_store=vector_store)
index = VectorStoreIndex.from_documents(
    documents=documents,
    storage_context=storage_context,
    show_progress=True)
else:
    vector_store = MilvusVectorStore(
        uri=milvus_store, dim=dimensions,
        collection_name=args.embed)
    index = VectorStoreIndex.from_vector_store(
        vector_store)

query_engine = index.as_query_engine()
response = query_engine.query(
    "Which butterfly species was found "
    "most frequently in Tyrol in 2023?")
print(response)

```

12.6.2 Conclusion

We were impressed by how much can be achieved using the LlamaIndex library, even with the default settings. The entire source code of the RAG script just presented comprises only 100 lines of code, whereby we've also included command-line parameters for the LLM to be used and the new creation of the index for our tests.

We don't want you to believe that you can create an application with such simple tricks that can be used successfully in a production environment. There are several examples of how a chatbot that was quickly put together wasn't a great success. Problems with LLMs such as *bias* can't be ignored. In addition, the application should also be secured against *creative* user input that attempts to use the LLM for purposes other than the queries you require.

For a company-internal knowledge database, however, this could be a starting point for a successful application. There is no question that tests

should be run continuously to check the quality of the answers according to previously defined rules.